Eine kurze Einführung in die objektorientierte Programmierung mit Python

DLBDSOOFPP01_D

26. April 2026

Inhaltsverzeichnis

1 Einführung in OOP und Python **3**

2 Klassen und Objekte **3**
- 2.1 Das Konzept von `self` . . . 4
- 2.2 Konstruktoren und Initialisierung mit `__init__` . . . 4
- 2.3 Attribute: Instanzattribute und Klassenattribute . . . 5

3 Sichtbarkeit und Kapselung in Python **6**

4 Properties statt klassischer Getter und Setter **7**

5 Vererbung **8**
- 5.1 Mehrfachvererbung . . . 9
- 5.2 Method Resolution Order und `super()` . . . 9
- 5.3 Abstrakte Klassen . . . 10

6 Polymorphie und Duck Typing **11**

7 Hat-ein-Beziehungen: Komposition und Aggregation **12**
- 7.1 Komposition . . . 13
- 7.2 Aggregation . . . 13

8 Spezielle Methoden **14**

9 Statische Methoden und Klassenmethoden **15**

10 Dataclasses **16**

11 Typannotationen **17**

12 Beispielprojekt **18**

13 Typische Fehler und Missverständnisse **20**
13.1 Zu viele Getter und Setter 20
13.2 Zu starke Formalisierung 20
13.3 Verwechslung von Klassenattributen und Instanzattributen 21
13.4 Überladene Konstruktoren 22
13.5 Übertragung von Denkweisen aus anderen Sprachen 23
13.6 Pythonischer Stil 23

14 Zusammenfassung **24**

1 Einführung in OOP und Python

Objektorientierte Programmierung, kurz OOP, ist ein Programmierparadigma, bei dem Programme durch das Zusammenspiel von Objekten beschrieben werden. Ein Objekt bündelt Daten und Verhalten. Daten werden meist durch Attribute repräsentiert, Verhalten durch Methoden.
Python eignet sich gut für den Einstieg in die objektorientierte Programmierung, da die Syntax im Vergleich zu vielen anderen Sprachen relativ kompakt und gut lesbar ist. Gleichzeitig handelt es sich bei Python nicht um eine rein objektorientierte Sprache, sondern um eine Mehrparadigmensprache. Neben objektorientierten Konzepten können auch prozedurale und funktionale Ansätze genutzt werden.
Dies ermöglicht es, die Prinzipien der objektorientierten Programmierung nicht isoliert zu betrachten, sondern im Kontext alternativer Lösungsansätze einzuordnen. Dadurch wird deutlich, dass die Wahl eines geeigneten Programmierparadigmas stets vom jeweiligen Problem und Anwendungskontext abhängt.
OOP ist kein Selbstzweck. Sie wird dann nützlich, wenn reale oder fachliche Strukturen in Software modelliert werden sollen. In einem Studienplan könnte es beispielsweise Objekte wie „Studiengang“, „Semester“, „Modul“ und „Prüfungsleistung“ geben. Jedes dieser Objekte hat eigene Eigenschaften und eigenes Verhalten.

2 Klassen und Objekte

Eine Klasse definiert die Struktur und das Verhalten von Objekten, indem sie festlegt, welche Attribute und Methoden diesen zur Verfügung stehen. Sie bildet damit einen abstrakten Typ, der die gemeinsamen Eigenschaften gleichartiger Objekte beschreibt.
Ein Objekt ist eine konkrete Instanz einer Klasse und repräsentiert eine spezifische Ausprägung dieses Typs. Beispielsweise beschreibt die Klasse „Modul“ allgemeine Eigenschaften eines Moduls, während „Mathematik 1“ eine konkrete Instanz dieser Klasse darstellt.
Zur Veranschaulichung kann eine Klasse auch als Bauplan verstanden werden, während ein Objekt eine konkrete Ausführung dieses Bauplans darstellt.

```python
class Modul:
    def __init__(self, titel, ects):
        self.titel = titel
        self.ects = ects

modul1 = Modul("Mathematik 1", 5)
print(modul1.titel)
print(modul1.ects)
```

Im Beispiel wird zunächst die Klasse `Modul` definiert. Der Konstruktor `__init__` legt fest, welche Daten beim Erzeugen eines Objekts übergeben werden müssen. Anschließend wird mit `Modul("Mathematik 1", 5)` ein konkretes Objekt erzeugt.
Wichtig ist die begriffliche Trennung:

- Die Klasse ist die allgemeine Beschreibung.

- Das Objekt ist die konkrete Ausprägung.
- Attribute speichern Zustände.
- Methoden beschreiben Verhalten.

2.1 Das Konzept von `self`

In Python werden Instanzmethoden mit einem ersten Parameter definiert, der konventionell `self` heißt.

```
class Begruessung:
    def hallo(self, name):
        print(f"Hallo {name}, ich bin ein Objekt der Klasse Begruessung.")

b = Begruessung()
b.hallo("Anna")
```

`self` wirkt zunächst ungewohnt. Technisch verweist `self` auf das Objekt, auf dem die Methode aufgerufen wird. Dadurch kann die Methode auf die Attribute genau dieses Objekts zugreifen.
Technisch gesehen ist `self` kein Spezialwort der Sprache, sondern eine Konvention. Theoretisch könnte auch ein anderer Name verwendet werden. Praktisch sollte man immer `self` verwenden, weil dies dem Python-Standard entspricht.

2.2 Konstruktoren und Initialisierung mit `__init__`

Der Konstruktor in Python ist nicht direkt `__init__`, sondern die Erzeugung eines Objekts läuft intern über `__new__` und danach über `__init__`. Für die Grundlagen genügt aber die Sichtweise, dass `__init__` die Initialisierung des Objekts übernimmt.

```
class Semester:
    def __init__(self, nummer):
        self.nummer = nummer
        self.module = []

    def modul_hinzufuegen(self, modul):
        self.module.append(modul)

semester1 = Semester(1)
semester1.modul_hinzufuegen("Physik")
print(semester1.module)
```

Der Konstruktor sorgt dafür, dass ein neues Objekt in einen gültigen Anfangszustand versetzt wird. Ein `Semester` besitzt im Beispiel direkt eine Nummer und eine leere Liste von Modulen. Ein wichtiger Gestaltungsgrundsatz lautet: Der Konstruktor sollte ein Objekt in einen konsistenten Zustand bringen. Gleichzeitig sollte er nicht mit zu viel technischer Logik überladen werden. Datenbankzugriffe, Dateizugriffe oder komplexe Geschäftslogik gehören meist nicht in den Konstruktor.

2.3 Attribute: Instanzattribute und Klassenattribute

Python unterscheidet zwischen Instanzattributen und Klassenattributen.

- Instanzattribute gehören zu einem konkreten Objekt. Zwei verschiedene Objekte derselben Klasse können unterschiedliche Werte dieser Attribute besitzen.
- Klassenattribute gehören zur Klasse selbst. Sie werden von allen Instanzen gemeinsam genutzt, solange sie nicht auf Instanzebene überschrieben werden.

```
class Modul:
    hochschule = "IUBH"

    def __init__(self, titel):
        self.titel = titel

m1 = Modul("Programmierung")
m2 = Modul("Datenbanken")

print(m1.hochschule)
print(m2.hochschule)

Modul.hochschule = "IU Internationale Hochschule"
print(m1.hochschule)
```

Das Beispiel zeigt, dass das Attribut `hochschule` an der Klasse definiert wurde. Alle Instanzen greifen darauf zu. Wird es an der Klasse verändert, sehen alle Objekte den neuen Wert.

Typische Fehlerquelle: veränderliche Klassenattribute

Besonders problematisch sind veränderliche Klassenattribute, zum Beispiel Listen oder Dictionaries.

```
class FehlerhaftesSemester:
    module = []

    def __init__(self, nummer):
```

```
        self.nummer = nummer

s1 = FehlerhaftesSemester(1)
s2 = FehlerhaftesSemester(2)

s1.module.append("Mathematik")
print(s1.module)
print(s2.module)
```

Hier teilen sich alle Instanzen dieselbe Liste `module`. Das ist oft nicht beabsichtigt. Für Anfänger ist das eine der häufigsten Stolperstellen in Python. Wenn jedes Objekt seine eigene Liste besitzen soll, muss diese im Konstruktor als Instanzattribut angelegt werden.

3 Sichtbarkeit und Kapselung in Python

Kapselung beschreibt den Schutz des inneren Zustands eines Objekts sowie den kontrollierten Zugriff von außen. Im Unterschied zu Sprachen wie Java oder C++ erzwingt Python dabei keine Zugriffsbeschränkungen, sondern folgt bewusst dem Prinzip der Konvention.

```
class Konto:
    def __init__(self, kontonummer, kontostand):
        self.kontonummer = kontonummer      # öffentlich
        self._kontostand = kontostand       # intern gedacht
        self.__pin = 1234                   # Name Mangling

konto = Konto("DE123", 100)

print(konto.kontonummer)
print(konto._kontostand)

# Zugriff auf __pin funktioniert so nicht:
# print(konto.__pin)

print(konto._Konto__pin)
```

Zur Kennzeichnung der beabsichtigten Sichtbarkeit haben sich drei Schreibweisen etabliert:

- `name`: öffentliches Attribut
- `_name`: als intern gedacht (Konvention, keine technische Einschränkung)
- `__name`: Name Mangling zur Vermeidung von Namenskonflikten, insbesondere bei Vererbung

Diese Unterscheidung stellt lediglich eine Annäherung an die in UML üblichen Sichtbarkeiten (`+`, `#`, `-`) dar. Eine direkte Übertragung ist nicht möglich, da Python keine Zugriffsbeschränkungen auf Sprachebene implementiert.
In der Praxis werden häufig öffentliche Attribute verwendet, insbesondere bei einfachen Datenklassen. Ein einzelner Unterstrich dient dazu, interne Implementierungsdetails zu kennzeichnen und eine klare Trennung zur öffentlichen Schnittstelle zu signalisieren. Das doppelte Unterstrich-Präfix wird dagegen gezielt und vergleichsweise selten eingesetzt.
Soll der Zugriff stärker kontrolliert werden, kann die Konvention `_name` mit `@property`-Methoden kombiniert werden. So lässt sich eine klare Schnittstelle definieren, ohne die Flexibilität von Python einzuschränken.
UML-Diagramme beschreiben die beabsichtigte Kapselung auf konzeptioneller Ebene. Die konkrete Umsetzung im Python-Code kann davon bewusst abweichen. Python verhindert also keinen Zugriff, sondern macht ihn durch Konventionen transparent. Dieses Verständnis wird häufig mit dem Motto „We are all consenting adults here“ beschrieben.

4 Properties statt klassischer Getter und Setter

In Java sind Getter und Setter weit verbreitet. In Python wird direkter Attributzugriff zunächst als normal angesehen. Erst wenn zusätzliche Logik benötigt wird, kommt oft `@property` ins Spiel.

```python
class Modul:
    def __init__(self, ects):
        self.ects = ects

    @property
    def ects(self):
        return self._ects

    @ects.setter
    def ects(self, wert):
        if wert <= 0:
            raise ValueError("ECTS müssen positiv sein. Wert nicht geändert.")
        self._ects = wert

m = Modul(5)
print(m.ects)
m.ects = 6
print(m.ects)
try:
    m.ects = -3
except ValueError as e:
    print(f"Fehler: {e}")
print(m.ects)
```

Der Vorteil von Properties liegt darin, dass sich aus Sicht des aufrufenden Codes nichts ändert. Das Attribut wird weiterhin wie ein Attribut benutzt, intern können aber Prüfungen, Berechnungen oder andere Mechanismen ablaufen.

5 Vererbung

Vererbung erlaubt es, eine neue Klasse auf Basis einer vorhandenen Klasse zu definieren. Die abgeleitete Klasse übernimmt Eigenschaften und Methoden der Basisklasse und kann sie erweitern oder überschreiben.

```
class Person:
    def __init__(self, name, alter):
        self.name = name
        self.alter = alter

    def vorstellen(self):
        print(f"Ich heiße {self.name}.")

    def nenne_alter(self):
        print(f"Ich bin {self.alter} Jahre alt.")

class Student(Person):
    def __init__(self, name, alter, matrikelnummer):
        super().__init__(name, alter)
        self.matrikelnummer = matrikelnummer

    def vorstellen(self):
        print(f"Ich heiße {self.name} und meine Matrikelnummer ist
        ↪  {self.matrikelnummer}.")

s = Student("Mia", 25, "123456")
s.vorstellen()
s.nenne_alter()
```

Im Beispiel erbt die Klasse `Student` von `Person`. Die Methode `vorstellen` wird überschrieben. Das ist ein typischer Fall von Spezialisierung: Jeder Student ist eine Person, besitzt aber zusätzliche Eigenschaften.
Häufig soll die bestehende Funktionalität der Basisklasse jedoch nicht vollständig ersetzt, sondern ergänzt werden. Dafür wird in Python die Funktion `super()` verwendet. Sie ermöglicht den Zugriff auf die entsprechende Methode der Basisklasse.
Typischerweise wird dabei zunächst die Methode der Basisklasse aufgerufen und anschließend eigenes Verhalten ergänzt.
Vererbung ist nützlich, wenn wirklich eine fachliche „ist-ein“-Beziehung vorliegt. Sie sollte nicht allein aus Bequemlichkeit eingesetzt werden. Zu tiefe Vererbungshierarchien machen Programme schnell unübersichtlich.

5.1 Mehrfachvererbung

Python erlaubt Mehrfachvererbung. Eine Klasse kann also von mehreren Basisklassen gleichzeitig erben.

```
class Speicherbar:
    def speichern(self):
        print("Objekt wird gespeichert.")

class Druckbar:
    def drucken(self):
        print("Objekt wird gedruckt.")

class Bericht(Speicherbar, Druckbar):
    pass

bericht = Bericht()
bericht.speichern()
bericht.drucken()
```

Das kann praktisch sein, etwa wenn verschiedene Fähigkeiten kombiniert werden sollen. Gleichzeitig erhöht Mehrfachvererbung die Komplexität. Besonders dann, wenn mehrere Basisklassen Methoden mit gleichem Namen besitzen, wird das Auflösungsverhalten wichtig. Für die Grundlagen sollte man Mehrfachvererbung kennen, aber zurückhaltend einsetzen. In vielen Fällen ist die Verwendung von Kompositionen einfacher und klarer.

5.2 Method Resolution Order und `super()`

Wenn mehrere Klassen in einer Vererbungshierarchie beteiligt sind, muss Python festlegen, in welcher Reihenfolge Methoden gesucht werden. Diese Reihenfolge wird als Method Resolution Order (MRO) bezeichnet.

```
class A:
    def aktion(self):
        print("A")

class B(A):
    def aktion(self):
        print("B")
        super().aktion()

class C(A):
    def aktion(self):
        print("C")
```

```
        super().aktion()

class D(B, C):
    def aktion(self):
        print("D")
        super().aktion()

d = D()
d.aktion()
print(D.mro())
```

Die MRO wird insbesondere bei Mehrfachvererbung relevant. Python verwendet hierfür eine konsistente Regel, die als C3-Linearisierung bezeichnet wird.
Für das Verständnis ist entscheidend, dass `super()` nicht einfach die direkte Elternklasse anspricht, sondern die nächste Klasse in dieser festgelegten Auflösungsreihenfolge.
Damit wird Vererbung in Python als kooperativer Mechanismus umgesetzt: Mehrere Klassen können durch die Verwendung von `super()` gemeinsam an der Ausführung einer Methode beteiligt sein, indem jede Klasse ihren eigenen Anteil ergänzt und den Aufruf anschließend entlang der MRO weitergibt.

5.3 Abstrakte Klassen

Im Kontext der Vererbung stellt sich häufig die Frage, wie sichergestellt werden kann, dass bestimmte Methoden in abgeleiteten Klassen vorhanden sind. Hierfür bieten abstrakte Klassen eine Möglichkeit. Sie legen fest, welche Methoden in konkreten Unterklassen implementiert werden müssen.

```
from abc import ABC, abstractmethod

class Bewertbar(ABC):
    @abstractmethod
    def note_berechnen(self):
        pass

class Klausur(Bewertbar):
    def __init__(self, punkte):
        self.punkte = punkte

    def note_berechnen(self):
        return 1.0 if self.punkte >= 96 else 1.3

k = Klausur(93)
print(k.note_berechnen())
```

Eine abstrakte Klasse kann nicht direkt instanziiert werden, sondern dient als Vorlage für abgeleitete Klassen. Sie legt fest, welche Methoden von diesen Klassen implementiert werden müssen. Im Unterschied dazu stellt eine normale Basisklasse bereits konkrete Funktionalität bereit, die übernommen oder überschrieben werden kann. Abstrakte Klassen ergänzen dies um verbindliche Anforderungen und strukturieren so eine Vererbungshierarchie. In der Praxis wird aber häufig ohne abstrakte Klassen gearbeitet. Entscheidend ist weniger die formale Struktur als vielmehr, dass Objekte das benötigte Verhalten bereitstellen.

6 Polymorphie und Duck Typing

Polymorphie bedeutet, dass unterschiedliche Objekte auf die gleiche Weise verwendet werden können, solange sie das benötigte Verhalten bereitstellen. In Python wird dies häufig durch sogenanntes Duck Typing umgesetzt. Dabei ist nicht entscheidend, zu welcher Klasse ein Objekt gehört, sondern ob es die benötigten Methoden besitzt. Diese Idee wird oft durch den Satz „Wenn es aussieht wie eine Ente, läuft wie eine Ente und quakt wie eine Ente, dann behandeln wir es wie eine Ente.“ veranschaulicht.

```python
class Klausur:
    def berechne_note(self):
        return 1.7


class Projekt:
    def berechne_note(self):
        return 2.3


# Gute Variante: Duck Typing
def drucke_note_gut(pruefungsleistung):
    print(pruefungsleistung.berechne_note())


# Schlechte Variante: Typprüfung
def drucke_note_schlecht(pruefungsleistung):
    if isinstance(pruefungsleistung, Klausur):
        print(pruefungsleistung.berechne_note())
    elif isinstance(pruefungsleistung, Projekt):
        print(pruefungsleistung.berechne_note())
    else:
        print("Unbekannte Prüfungsleistung")


leistungen = [
    Klausur(),
```

```
    Projekt(),
]


print("Gute Variante:")
for leistung in leistungen:
    drucke_note_gut(leistung)


print("\nSchlechte Variante:")
for leistung in leistungen:
    drucke_note_schlecht(leistung)


# Erweiterung: neue Klasse
class Hausarbeit:
    def berechne_note(self):
        return 1.3


print("\nNeue Klasse hinzugefügt:")

print("Gute Variante:")
drucke_note_gut(Hausarbeit())

print("Schlechte Variante:")
drucke_note_schlecht(Hausarbeit())
```

Im Beispiel sind zwei Varianten der Funktion `drucke_note` implementiert. Die erste Variante nutzt Duck Typing und funktioniert unabhängig von der konkreten Klasse. Die zweite Variante unterscheidet explizit zwischen Klassen und ist dadurch weniger flexibel.
Der Vorteil von Polymorphie zeigt sich insbesondere bei Erweiterungen: Neue Klassen können hinzugefügt werden, ohne bestehenden Code anpassen zu müssen.

7 Hat-ein-Beziehungen: Komposition und Aggregation

Neben Vererbungsbeziehungen, die „ist-ein“-Beziehungen beschreiben (z. B. „Ein Student ist eine Person“), gibt es in der objektorientierten Modellierung auch „hat-ein“-Beziehungen. Dabei geht es nicht um Spezialisierung, sondern darum, dass ein Objekt andere Objekte enthält oder verwendet.

7.1 Komposition

Komposition beschreibt eine enge „hat-ein“-Beziehung. Die enthaltenen Objekte sind Teil der internen Struktur des übergeordneten Objekts und werden typischerweise nicht unabhängig verwendet.
Ein einfaches Beispiel ist ein Haus, das aus mehreren Räumen besteht.

```
class Raum:
    def __init__(self, name):
        self.name = name


class Haus:
    def __init__(self):
        self.raeume = []
        self.raeume.append(Raum("Küche"))
        self.raeume.append(Raum("Wohnzimmer"))
        self.raeume.append(Raum("Schlafzimmer"))


haus = Haus()

for raum in haus.raeume:
    print(raum.name)
```

Im Beispiel werden die `Raum`-Objekte innerhalb des `Haus`-Objekts erzeugt. Dadurch wird deutlich, dass die Räume als feste Bestandteile des Hauses modelliert sind.

7.2 Aggregation

Aggregation beschreibt eine lockerere „hat-ein“-Beziehung. Die enthaltenen Objekte existieren unabhängig und können auch in anderen Zusammenhängen verwendet werden.
Ein Beispiel ist die Beziehung zwischen einem Raum und Möbeln. Möbel existieren unabhängig vom Raum und können in verschiedenen Räumen verwendet werden.

```
class Moebel:
    def __init__(self, name):
        self.name = name


class Raum:
    def __init__(self, name):
        self.name = name
        self.moebel = []
```

```
10
11      def moebel_hinzufuegen(self, moebel):
12          self.moebel.append(moebel)
13
14
15  # Möbel werden unabhängig erzeugt
16  tisch = Moebel("Tisch")
17  stuhl = Moebel("Stuhl")
18
19  wohnzimmer = Raum("Wohnzimmer")
20  wohnzimmer.moebel_hinzufuegen(tisch)
21  wohnzimmer.moebel_hinzufuegen(stuhl)
22
23  # Dasselbe Möbelstück könnte auch in einem anderen Raum stehen
24  arbeitszimmer = Raum("Arbeitszimmer")
25  arbeitszimmer.moebel_hinzufuegen(tisch)
26
27  for m in wohnzimmer.moebel:
28      print(m.name)
```

Im Beispiel werden die `Moebel`-Objekte unabhängig erzeugt und anschließend einem Raum hinzugefügt. Dass ein Möbelstück auch in mehreren Räumen verwendet werden kann, ist ein typisches Merkmal einer Aggregation.

Abgrenzung

Der Unterschied zwischen Komposition und Aggregation liegt in der Stärke der Bindung zwischen den Objekten:

- Bei der Komposition sind die enthaltenen Objekte fester Bestandteil des übergeordneten Objekts.
- Bei der Aggregation bleiben die enthaltenen Objekte unabhängig und können auch in anderen Kontexten verwendet werden.

In Python wird diese Unterscheidung nicht durch die Sprache erzwungen, sondern ergibt sich aus der fachlichen Modellierung und der Art, wie Objekte erzeugt und verwendet werden. Im Studiengangskontext ist die Abgrenzung zwischen Komposition und Aggregation oft nicht eindeutig. Welche Modellierung gewählt wird, hängt vom fachlichen Fokus und der gewünschten Verwendung ab.

8 Spezielle Methoden

Python besitzt zahlreiche spezielle Methoden, die oft auch als Magic Methods oder Dunder Methods bezeichnet werden, weil sie doppelte Unterstriche am Anfang und Ende tragen.

```python
class Modul:
    def __init__(self, titel, ects):
        self.titel = titel
        self.ects = ects

    def __str__(self):
        return f"{self.titel} ({self.ects} ECTS)"

    def __repr__(self):
        return f"Modul(titel={self.titel!r}, ects={self.ects!r})"

m = Modul("Programmierung", 5)
print(str(m))
print(repr(m))
```

Sie ermöglichen die Integration eigener Klassen in die Sprache selbst. Durch `__str__` lässt sich etwa festlegen, wie ein Objekt für Benutzer lesbar ausgegeben wird. `__repr__` soll eher eine präzise, entwicklerorientierte Darstellung liefern.
Weitere wichtige Beispiele sind:

- `__len__` für die Länge eines Objekts
- `__eq__` für Vergleiche
- `__lt__` für Ordnungsrelationen
- `__iter__` für Iteration
- `__contains__` für `in`

Damit zeigt sich ein wichtiges Merkmal von Python: OOP ist eng mit der allgemeinen Sprachgestaltung verzahnt.

9 Statische Methoden und Klassenmethoden

Bisher wurden Methoden betrachtet, die auf einem konkreten Objekt arbeiten und daher Zugriff auf dessen Zustand besitzen. In vielen Fällen ist dies ausreichend. Es gibt jedoch Situationen, in denen kein konkretes Objekt vorliegt oder benötigt wird. In solchen Fällen bietet Python alternative Methodenarten an.

```python
class Modul:
    faktor = 30

    def __init__(self, ects):
        self.ects = ects
```

```
6
7       def arbeitsstunden(self):
8           return self.ects * self.faktor
9
10      @classmethod
11      def mit_standard_ects(cls):
12          return cls(5)
13
14      @staticmethod
15      def ist_gueltiger_titel(titel):
16          return titel.strip() != ""
17
18  m = Modul.mit_standard_ects()
19  print(m.arbeitsstunden())
20  print(Modul.ist_gueltiger_titel("Physik"))
```

Im Beispiel wird eine Klassenmethode verwendet, um ein Objekt mit Standardwerten zu erzeugen. Dies ist sinnvoll, da noch kein Objekt existiert, auf dem eine Instanzmethode aufgerufen werden könnte. Die statische Methode prüft hingegen, ob ein Titel gültig ist. Diese Funktion benötigt weder Zugriff auf ein konkretes Objekt noch auf die Klasse selbst, gehört aber fachlich zum Kontext der Klasse. Die Unterschiede lassen sich wie folgt zusammenfassen:

- Instanzmethoden beschreiben Verhalten eines konkreten Objekts.
- Klassenmethoden beschreiben Verhalten, das sich auf die Klasse als Ganzes bezieht.
- Statische Methoden kapseln allgemeine Funktionen, die thematisch zur Klasse gehören.

In der Praxis werden Instanzmethoden am häufigsten verwendet. Klassenmethoden und statische Methoden kommen vor allem in speziellen Situationen zum Einsatz.

10 Dataclasses

Bei vielen Klassen besteht ein großer Teil des Codes darin, Attribute zu definieren und grundlegende Methoden wie einen Konstruktor oder eine textuelle Darstellung eines Objekts bereitzustellen. Dieser sogenannte Boilerplate-Code trägt wenig zur eigentlichen Fachlogik bei. Python bietet mit dem Modul `dataclasses` eine Möglichkeit, diesen Code automatisch zu erzeugen.

```
1  from dataclasses import dataclass
2
3  @dataclass
4  class Modul:
5      titel: str
6      ects: int
```

```
    pflicht: bool = True

m = Modul("Elektrotechnik", 5)
print(m)
```

Mit dem Dekorator `@dataclass` werden unter anderem ein Konstruktor sowie sinnvolle Standardmethoden wie `__repr__` automatisch generiert. Dadurch kann sich der Code stärker auf die fachliche Modellierung konzentrieren.
Dataclasses sind besonders nützlich für einfache Datenobjekte, bei denen hauptsächlich Attribute verwaltet werden, etwa Module, Semester oder andere Elemente eines Domänenmodells. Sie ersetzen jedoch nicht jede normale Klasse. Sobald komplexe Initialisierungslogik, umfangreiches Verhalten oder spezielle Kontrollmechanismen erforderlich sind, kann eine klassische Klasse besser geeignet sein.

11 Typannotationen

Python ist dynamisch typisiert. Das bedeutet, dass Variablen keinen festen Typ besitzen, sondern zur Laufzeit unterschiedliche Werte annehmen können. Im Gegensatz dazu müssen in statisch typisierten Sprachen wie Java oder C++ die Typen von Variablen bereits zur Übersetzungszeit festgelegt werden. In Python können dennoch Typannotationen angegeben werden. Dabei handelt es sich um Zusatzinformationen im Code, die beschreiben, welche Typen erwartet werden, ohne das Verhalten des Programms direkt zu verändern.

```
class Modul:
    def __init__(self, titel: str, ects: int) -> None:
        self.titel: str = titel
        self.ects: int = ects

    def arbeitsstunden(self) -> int:
        return self.ects * 30

m = Modul("Informatik", 6)
print(m.arbeitsstunden())
```

Typannotationen haben mehrere Vorteile:

- Sie verbessern die Lesbarkeit des Codes.
- Werkzeuge wie `mypy` oder moderne Entwicklungsumgebungen können damit Prüfungen und Hinweise geben.
- Sie helfen bei der Dokumentation von Schnittstellen.

Typannotationen sind in Python in der Regel optional und werden zur Laufzeit nicht automatisch überprüft. Sie dienen daher in erster Linie der besseren Verständlichkeit und der Unterstützung durch Werkzeuge.

12 Beispielprojekt

Im Folgenden werden einige der zuvor eingeführten Konzepte kombiniert.

```python
from dataclasses import dataclass, field

@dataclass
class Pruefungsleistung:
    name: str
    punkte: float
    max_punkte: float

    @property
    def prozent(self) -> float:
        if self.max_punkte == 0:
            return 0.0
        return self.punkte / self.max_punkte * 100


@dataclass
class Modul:
    titel: str
    ects: int
    pruefungen: list[Pruefungsleistung] = field(default_factory=list)

    def pruefung_hinzufuegen(self, pruefung: Pruefungsleistung) -> None:
        self.pruefungen.append(pruefung)

    def durchschnitt(self) -> float:
        if not self.pruefungen:
            return 0.0
        return sum(p.prozent for p in self.pruefungen) / len(self.pruefungen)


@dataclass
class Semester:
    nummer: int
    module: list[Modul] = field(default_factory=list)

    def modul_hinzufuegen(self, modul: Modul) -> None:
        self.module.append(modul)

```

```python
@dataclass
class Studiengang:
    name: str
    semester: list[Semester] = field(default_factory=list)

    def semester_hinzufuegen(self, semester: Semester) -> None:
        self.semester.append(semester)


# Module werden unabhängig erzeugt (Aggregation!)
mathe = Modul("Mathematik 1", 5)
physik = Modul("Physik 1", 5)

mathe.pruefung_hinzufuegen(Pruefungsleistung("Klausur", 82, 100))
physik.pruefung_hinzufuegen(Pruefungsleistung("Klausur", 75, 100))

semester1 = Semester(1)
semester1.modul_hinzufuegen(mathe)
semester1.modul_hinzufuegen(physik)

studiengang = Studiengang("Elektrotechnik")
studiengang.semester_hinzufuegen(semester1)

print(f"Studiengang: {studiengang.name}")
print(f"Mathe-Durchschnitt: {mathe.durchschnitt():.2f} %")
```

Das Beispiel modelliert einen Studiengang mit Semestern, Modulen und Prüfungsleistungen. Dabei werden mehrere zentrale Aspekte der objektorientierten Programmierung sichtbar:

- **Dataclasses** werden verwendet, um Datenstrukturen kompakt zu definieren und Boilerplate-Code zu vermeiden.

  Hierbei ist die Verwendung von `field(default_factory=list)` zu beachten, die sicher stellt, dass jede Instanz eine eigene Liste erhält und keine unbeabsichtigte gemeinsame Nutzung entsteht.

- **Komposition und Aggregation** modellieren unterschiedliche fachliche Beziehungen: Ein Studiengang enthält Semester im Sinne einer Komposition. Module hingegen werden unabhängig erzeugt und anschließend Semestern zugeordnet, was einer Aggregation entspricht.

- **Properties** werden genutzt, um berechnete Werte wie den prozentualen Anteil einer Prüfungsleistung bereitzustellen.

- **Typannotationen** machen die erwarteten Datentypen explizit und verbessern die Lesbarkeit.

- **Methoden** kapseln fachliche Logik, etwa das Hinzufügen von Elementen oder die Berechnung eines Durchschnitts.

Das Beispiel zeigt, dass objektorientierte Konzepte nicht isoliert auftreten, sondern gemeinsam ein konsistentes fachliches Modell ermöglichen.

13 Typische Fehler und Missverständnisse

Auch im vorangegangenen Beispielprojekt zeigen sich typische Stolperstellen, die insbesondere bei der Arbeit mit objektorientierten Konzepten in Python auftreten können. Die folgenden Beispiele zeigen typische Fehlmuster und deren einfachere Alternativen.

13.1 Zu viele Getter und Setter

Direkter Attributzugriff ist in Python oft völlig ausreichend. Getter werden erst dann sinnvoll, wenn zusätzliche Logik notwendig ist.

```
class Modul:
    def __init__(self, titel):
        self.titel = titel

    # Getter wäre hier unnötig
    def get_titel(self):
        return self.titel


m = Modul("Physik")

# Direkter Zugriff ist üblich
print(m.titel)

# Zugriff über Getter funktioniert, ist aber oft unnötig
print(m.get_titel())
```

13.2 Zu starke Formalisierung

Nicht jede Problemstellung erfordert eine formale Klassenhierarchie mit Vererbung oder abstrakten Basisklassen. Insbesondere bei einfachen Anforderungen kann dies den Code unnötig komplex machen.

```
# unnötig formalisiert
class Fahrzeug:
    def fahren(self):
        raise NotImplementedError
```

```python


class Auto(Fahrzeug):
    def fahren(self):
        print("Auto fährt")


def starte_fahrt(fahrzeug: Fahrzeug):
    fahrzeug.fahren()


starte_fahrt(Auto())


# einfacher und ausreichend
class Auto_einfach:
    def fahren(self):
        print("Auto fährt")


def starte_fahrt_einfach(objekt):
    objekt.fahren()


starte_fahrt_einfach(Auto_einfach())
```

Im Beispiel wird zunächst eine abstrakte Basisklasse verwendet, obwohl diese keinen echten Mehrwert bietet. Die einfachere Variante verzichtet darauf und nutzt direkt das benötigte Verhalten.
Dies verdeutlicht, dass in Python häufig weniger formale Strukturen ausreichen, solange die benötigten Methoden vorhanden sind.

13.3 Verwechslung von Klassenattributen und Instanzattributen

Insbesondere bei veränderlichen Objekten wie Listen führt dies zu unerwartetem Verhalten.

```python
class Modul:
    pruefungen = []  # gemeinsames Attribut!


m1 = Modul()
m2 = Modul()

```

```
m1.pruefungen.append("Klausur")

# überraschend: auch m2 enthält den Eintrag
print(m2.pruefungen)
```

13.4 Überladene Konstruktoren

Wenn Konstruktoren zu viel Logik enthalten, wird der Code schwer verständlich und wartbar.

```
# Bad: Überladener Konstruktor
class Modul:
    def __init__(self, titel, ects):
        self.titel = titel
        self.ects = ects

        # Validierung im Konstruktor
        if ects <= 0:
            raise ValueError("ECTS müssen positiv sein")

        # Berechnung im Konstruktor
        self.arbeitsstunden = ects * 30


# Good: Klare Trennung von Initialisierung und Verhalten
class ModulEinfach:
    def __init__(self, titel, ects):
        self.titel = titel
        self.ects = ects

    def berechne_arbeitsstunden(self):
        return self.ects * 30


# Verwendung
m_bad = Modul("Mathematik", 5)
print(m_bad.arbeitsstunden)

m_good = ModulEinfach("Mathematik", 5)
print(m_good.berechne_arbeitsstunden())
```

13.5 Übertragung von Denkweisen aus anderen Sprachen

Python bevorzugt häufig einfache und direkte Lösungen. Unnötige Methoden, überflüssige Vergleiche oder zu ausführliche Kontrollstrukturen führen zu schwer lesbarem Code.

```
# Unnötige Methoden und Vergleiche
class Modul:
    def __init__(self, titel):
        self.titel = titel

    def hat_titel(self):
        if self.titel is None:
            return False
        else:
            return True


m = Modul("Physik")

if m.hat_titel() == True:
    print("Titel vorhanden")


# Pythonischer Stil: direkte und knappe Lösung
class ModulEinfach:
    def __init__(self, titel):
        self.titel = titel


m2 = ModulEinfach("Physik")

if m2.titel:
    print("Titel vorhanden")
```

13.6 Pythonischer Stil

Die zuvor dargestellten Beispiele und typischen Fehler lassen sich auf gemeinsame Gestaltungsprinzipien zurückführen, die als pythonischer Stil bezeichnet werden.
Zentrale Merkmale sind:

- einfache und direkte Lösungen statt unnötiger Komplexität
- Lesbarkeit vor formaler Strenge
- Nutzung vorhandener Sprachmittel statt eigener Konstruktionen

- Orientierung am tatsächlichen Problem statt an abstrakten Mustern
- Verwendung von `snake_case` für Variablen- und Funktionsnamen sowie Methoden
- Verwendung von `CamelCase` für Klassennamen

Diese Prinzipien zeigen sich nicht in einzelnen Sprachkonstrukten, sondern in der Gesamtheit des Codes.

14 Zusammenfassung

Die objektorientierte Programmierung in Python folgt denselben Grundideen wie in anderen Sprachen, wird jedoch deutlich weniger formal umgesetzt. Im Mittelpunkt steht nicht die strikte Einhaltung von Strukturen, sondern eine klare und problemorientierte Modellierung. Zentrale Aspekte sind dabei:

- Klassen dienen der Strukturierung von Daten und Verhalten, ohne dass strikte Zugriffsbeschränkungen oder Hierarchien erzwungen werden.
- Objekte werden in der Regel direkt verwendet, wobei Attributzugriff und Methodenaufrufe bewusst einfach gehalten sind.
- Polymorphie wird häufig über Duck Typing realisiert, sodass das Verhalten eines Objekts wichtiger ist als seine konkrete Klassenzugehörigkeit.
- Fachliche Beziehungen werden vorzugsweise über Komposition und Aggregation modelliert, während Vererbung gezielt eingesetzt wird.
- Dataclasses und Typannotationen unterstützen eine kompakte, lesbare und strukturierte Beschreibung von Datenmodellen.

Die betrachteten Beispiele und typischen Fehler zeigen, dass viele Probleme weniger aus fehlendem Wissen über Sprachkonstrukte entstehen, sondern aus ungeeigneten Entwurfsentscheidungen.
Objektorientierte Programmierung in Python bedeutet daher vor allem, einfache, verständliche und fachlich passende Lösungen zu entwickeln, anstatt formale Konzepte mechanisch zu übertragen.